Teste de Software

Trabalho: Detecção e Refatoração de Test Smells

Relatório escrito

Aluno: Miguel Figueiredo Diniz

Matrícula: 853959

Repositório: github.com/DevMiguelDiniz/test-smelly

1. Análise de Smells

A suíte original (`test/userService.smelly.test.js`) testa o `UserService` e concentra vários maus cheiros. Três deles são descritos abaixo.

1.1 Lógica Condicional no Teste (Conditional Test Logic)

O teste *deve desativar usuários se eles não forem administradores* usa um `for` com `if/else` para decidir quais asserts executar. **Por que é um mau cheiro:** um teste deve ser linear e determinístico, funcionando como especificação simples; laços e condicionais o tornam difícil de ler e fazem com que ele próprio possa conter bugs. Além disso, ele mistura duas regras de negócio (desativar usuário comum e proteger administrador). **Risco:** um `expect` pode nunca ser executado sem que ninguém perceba, e, quando falha, não fica claro qual regra foi violada.

1.2 Falso positivo com try/catch (Expected Exception Smell)

No teste *deve falhar ao criar usuário menor de idade*, a chamada a `createUser` fica dentro de um `try` e o assert dentro do `catch`. **Por que é um mau cheiro:** se a exceção não for lançada, o `catch` não executa, nenhum assert roda e o teste passa. **Risco:** é o pior cenário para um teste, pois dá falsa sensação de segurança. Se a validação de idade for removida do código, a suíte continua verde e o bug chega à produção.

1.3 Teste Frágil (Fragile Test)

O teste do relatório compara a saída com a string exata `ID: ..., Nome: Alice, Status: ativo\n` e com o cabeçalho completo. **Por que é um mau cheiro:** o teste está acoplado à formatação (espaços, vírgulas, quebras de linha) e não ao comportamento. O próprio código-fonte avisa que o formato pode mudar. **Risco:** mudanças cosméticas, sem nenhum defeito funcional, quebram o teste, gerando custo de manutenção e desconfiança na suíte. Outros smells observados foram Eager Test / Assertion Roulette (teste 1) e teste ignorado e vazio (`test.skip`), detalhados em `docs/analise-test-smells.md`.

2. Processo de Refatoração

O teste mais problemático foi o de desativação de usuários (lógica condicional + Eager Test). A refatoração foi feita em `__tests__/userService.clean.test.js`, sem alterar o arquivo original.

Antes (arquivo smelly):

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (arquivo clean):

```
describe('deactivateUser', () => {
  test('retorna true ao desativar um usuário comum', () => {
    // Arrange
    const comum = userService.createUser('Comum', 'comum@teste.com', 30);
    // Act
    const resultado = userService.deactivateUser(comum.id);
    // Assert
    expect(resultado).toBe(true);
  });

  test('marca um usuário comum como inativo após a desativação', () => {
    const comum = userService.createUser('Comum', 'comum@teste.com', 30);
    userService.deactivateUser(comum.id);
    expect(userService.getUserById(comum.id).status).toBe('inativo');
  });

  test('retorna false ao tentar desativar um administrador', () => {
    const admin = userService.createUser('Admin', 'admin@teste.com', 40, true);
    const resultado = userService.deactivateUser(admin.id);
    expect(resultado).toBe(false);
  });

  test('mantém o administrador ativo após tentativa de desativação', () => {
    const admin = userService.createUser('Admin', 'admin@teste.com', 40, true);
    userService.deactivateUser(admin.id);
    expect(userService.getUserById(admin.id).status).toBe('ativo');
  });
});  // (nos arquivos reais, cada teste segue Arrange / Act / Assert)
```

Decisões tomadas: (1) removi o `for` e o `if/else`, criando um cenário explícito para cada regra; (2) dividi o teste em quatro, cada um com um único motivo para falhar (retorno e estado, para comum e para administrador), eliminando o Eager Test; (3) apliquei Arrange/Act/Assert, de modo que cada teste prepara os dados, executa uma ação e verifica; (4) usei nomes que descrevem o comportamento esperado, de forma que a falha já indica o que quebrou; (5) agrupei por método com `describe`. Os demais smells foram tratados de forma análoga: o try/catch virou `expect(fn).toThrow(msg)`, o relatório passou a ser verificado por conteúdo (nome, id e status) em vez da linha exata, e o teste vazio foi implementado (relatório sem usuários). Também foram adicionados testes para casos antes não cobertos (id inexistente e nome obrigatório).

3. Relatório da Ferramenta (ESLint)

Foi configurado o ESLint 8 com `eslint-plugin-jest` (regras `jest/no-conditional-expect`, `jest/no-disabled-tests` e `jest/no-identical-title`, além do conjunto `jest/recommended`). Resultado da primeira execução de `npx eslint .`:

```
$ npx eslint .

/home/miguel-diniz/test-smelly/test/userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Disabled test                          jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
```

Figura 1: primeira execução do ESLint, com 4 erros e 2 avisos no arquivo smelly.

A ferramenta apontou automaticamente a lógica condicional (linhas 44, 46 e 49), o `expect` dentro do `catch` (linha 73), o teste desabilitado e o teste sem assertions (linha 77). Ou seja, três dos smells da análise manual foram detectados sem leitura humana, em segundos, e podem rodar a cada commit ou no CI. Por outro lado, o ESLint não detectou o teste frágil, o Eager Test nem o estado global compartilhado, que dependem de interpretação do significado do teste. Após a refatoração, `npx`

`eslint .` não reporta nada no arquivo limpo e `npm test` passa nos dois arquivos (21 testes passando e 1 ignorado, do arquivo original).

4. Conclusão

Testes limpos funcionam como documentação executável: são curtos, lineares, têm nome claro e falham por um único motivo, o que reduz o tempo de diagnóstico e o custo de manutenção. Testes com smells, ao contrário, geram falsos positivos, quebram sem necessidade e minam a confiança na suíte, até que ela deixe de ser usada. A análise estática complementa a revisão humana ao automatizar a detecção de padrões problemáticos e impor o padrão de forma consistente a toda a equipe, ainda no editor ou no pipeline. Ela não substitui o julgamento, já que smells semânticos como fragilidade e testes ansiosos ainda exigem revisão, mas, combinada com boas práticas como AAA e um comportamento por teste, ajuda a manter o projeto sustentável conforme ele cresce.